

Absolutely. Here is a **complete orientation/continuation document** you can copy into a new chat. It contains the project context, current architecture, what has been completed, important dataset findings, current problems, and the exact next steps.

MedAssist AI — Complete Project Orientation

1. Project

Project name: MedAssist AI

The project is being developed as a healthcare assistance system with:

- Patient functionality
- Healthcare/provider functionality planned
- Patient profile management
- Symptom management
- Disease prediction using ML
- Risk assessment
- Recommendations
- Database persistence
- Frontend + backend integration

The immediate objective is to complete **Milestone 1** and prepare for its presentation.

Important project constraint

Use only the **technologies, datasets, and modules explicitly specified in the project PDF**. Do not introduce unrelated technologies or features outside the project requirements.

2. Current Development Environment

Operating system

Windows

Backend

Python + FastAPI

The backend is located at:

```
MedAssist-AI/  
└─ backend/
```

The backend runs with:

```
uvicorn app.main:app --reload
```

Current development URL:

```
http://127.0.0.1:8000
```

Swagger documentation:

```
http://127.0.0.1:8000/docs
```

Virtual environment

The project uses a Python virtual environment:

```
venv/
```

Typical terminal:

```
(venv) PS C:\Users\Hemanth\Documents\Projects_2026\MedAssist-AI\backend>
```

Database

PostgreSQL

Database:

```
medassist_ai
```

Database user:

```
postgres
```

The current connection function is in:

```
backend/app/database.py
```

Current code:

```
import psycopg2  
  
def get_database_connection():  
    connection = psycopg2.connect(  
        dbname="medassist_ai",  
        user="postgres",  
        password="psql@2026",  
        host="localhost",
```

```
        port="5432"  
    )  
  
    return connection
```

3. Current Backend Structure

Current backend structure:

```
backend/  
├── requirements.txt  
└── app/  
    ├── database.py  
    ├── dataset_loader.py  
    ├── inspect_datasets.py  
    ├── main.py  
    ├── auth.py  
    ├── patient.py  
    ├── prediction_service.py  
    ├── train_disease_model.py  
    ├── test_prediction.py  
    ├── test_dataset_loader.py  
    ├── models/  
    │   ├── disease_prediction_model.pkl  
    │   └── feature_names.pkl  
    └── __init__.py
```

There may also be `__pycache__` files.

4. Backend Application

The FastAPI application is in:

```
backend/app/main.py
```

The application includes:

- Authentication router
- Patient router
- Dataset summary endpoint

The backend is currently successfully starting after resolving several import and database connection issues.

5. Authentication Completed

User authentication has been implemented.

The `users` table is:

```
users
-----
id
full_name
email
password_hash
role
created_at
```

Structure:

```
id            integer PRIMARY KEY
full_name     varchar(100) NOT NULL
email         varchar(150) UNIQUE NOT NULL
password_hash text NOT NULL
role          varchar(30) DEFAULT 'patient'
created_at    timestamp DEFAULT CURRENT_TIMESTAMP
```

Authentication includes:

- User registration
- Password hashing
- Login
- JWT authentication
- Protected endpoints using:

```
Depends (get_current_user)
```

Password hashing was initially implemented with `passlib[bcrypt]`.

There was a compatibility issue:

```
AttributeError:
module 'bcrypt' has no attribute '__about__'
```

and:

```
ValueError:
password cannot be longer than 72 bytes
```

This was resolved sufficiently for the authentication flow to work.

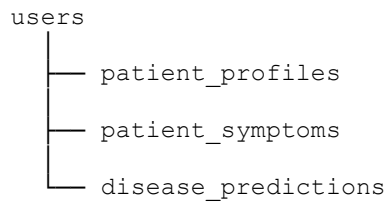
6. Current PostgreSQL Tables

The current database contains:

```
patient_profiles
patient_symptoms
users
```

A `disease_predictions` table was also created/used for prediction history.

The current conceptual database relationship is:



The patient owns all of these records through:

```
user_id
```

Important security principle:

Every patient-specific query must filter using the authenticated user's ID.

Example:

```
WHERE user_id = %s
```

This prevents one patient from accessing another patient's data.

7. Patient Module

The main patient API file is:

```
backend/app/patient.py
```

The clean version currently contains:

Patient profile

```
POST /patient/profile
```

Creates a patient profile.

Fields:

```
date_of_birth
```

```
gender
phone
blood_group
height_cm
weight_kg
emergency_contact_name
emergency_contact_phone
```

The API prevents creating duplicate profiles for the same user.

Add symptom

```
POST /patient/symptoms
```

Input:

```
{
  "symptom_name": "fever",
  "severity": "moderate"
}
```

The symptom is stored in PostgreSQL.

Get patient's symptoms

```
GET /patient/symptoms
```

Returns symptoms belonging only to the authenticated user.

Delete symptom

```
DELETE /patient/symptoms/{symptom_id}
```

The deletion is restricted to the current authenticated user.

Disease prediction

```
POST /patient/predict-disease
```

Input:

```
{
  "symptoms": [
```

```
    "fever"  
  ]  
}
```

The endpoint:

```
Patient symptoms  
  ↓  
Prediction service  
  ↓  
ML model  
  ↓  
Predicted disease  
  ↓  
Store prediction in PostgreSQL  
  ↓  
Return prediction
```

Example response:

```
{  
  "user_id": 1,  
  "symptoms": [  
    "fever"  
  ],  
  "predicted_disease": "..."  
}
```

Available symptoms

```
GET /patient/available-symptoms
```

This returns the symptom names used by the ML model.

The model currently has:

```
1326 feature names
```

8. Machine Learning Work Completed

The initial model is a:

```
RandomForestClassifier
```

The initial training script loaded:

```
trainings.csv
```

The model was saved as:

```
models/disease_prediction_model.pkl
```

Feature names were saved as:

```
models/feature_names.pkl
```

A test prediction successfully produced:

```
Predicted disease:  
Alpha-1 antitrypsin deficiency
```

The model was integrated into:

```
prediction_service.py
```

and then connected to:

```
patient.py
```

9. Major ML Problem Discovered

The current initial model is **not reliable enough**.

Examples observed:

```
Fever → Gout  
Low blood pressure → Epilepsy
```

This means the API is working, but the prediction model/data pipeline requires further investigation.

Important distinction:

```
API working ≠ ML model reliable
```

The prediction endpoint works technically, but the dataset/model must be validated before trusting the predictions.

10. Datasets Currently Available

There are three relevant datasets.

Dataset 1: trainings.csv

Current information:

```
Rows: 391  
Columns: 1327
```


Features: 1326
Target: Prognosis
Unique diseases: 391
Missing values: 0

Structure:

Symptom 1
Symptom 2
Symptom 3
...
Symptom 1326
Prognosis

Important finding:

391 rows
391 unique disease labels

This strongly suggests that the training dataset has approximately one example per disease.

This is a serious limitation for a normal multi-class classification model.

Dataset 2: testing.csv

Current information:

Approximately 414 data rows
1327 columns
1326 symptom features
Target column: Prognosis (prediction)

The testing dataset has the same basic feature structure as the training dataset.

The user confirmed:

Training and testing datasets have the same columns.

There are:

26 duplicate entries

The user is considering removing the duplicates and combining both datasets.

Dataset 3: Disease symptom and patient profile dataset

Uploaded file:

Disease_symptom_and_patient_profile_dataset(1).csv

Shape:

349 rows
10 columns

Columns:

Disease
Fever
Cough
Fatigue
Difficulty Breathing
Age
Gender
Blood Pressure
Cholesterol Level
Outcome Variable

Example features:

Symptoms:

- Fever
- Cough
- Fatigue
- Difficulty Breathing

Patient/health features:

- Age
- Gender
- Blood Pressure
- Cholesterol Level

Target:

- Outcome Variable

This dataset is fundamentally different from the 1326-feature disease-symptom datasets.

Do **not** automatically merge this dataset with `trainings.csv` and `testing.csv`.

They likely serve different purposes and must be mapped to the project requirements.

11. Dataset Investigation So Far

The following has already been established:

Training/testing disease overlap

Previously checked:

Training diseases: 391

Testing diseases: 389
Common diseases: 389

So two diseases appear in the training set but not the testing set.

This should be investigated.

12. Symptom Naming Problem

The 1326-feature dataset contains inconsistent symptom names.

Examples found:

```
Fever
fever
Fever-- usually low-grade
```

For fatigue:

```
Fatigue(Feeling Tired)
Fatigue, reduced exercise ability
```

For cold:

```
Cold arm or leg
Intolerance to cold
Cold feet or legs
Changes in skin temperature, switching between warm or cold
```

This creates a major problem.

If a user enters:

```
fatigue
```

but the dataset feature is:

```
Fatigue(Feeling Tired)
```

the exact feature match may fail.

Observed logs:

```
Activated symptom: fever
```

but:

```
Unknown symptom: cold
Unknown symptom: fatigue
```

Therefore, the project needs a proper symptom mapping/normalization approach.

Possible logical pipeline:

```
User symptom
      ↓
Normalize input
      ↓
Map to dataset feature
      ↓
Activate correct feature
      ↓
ML model
      ↓
Prediction
```

But the exact mapping strategy should be designed after the dataset audit.

13. Current Dataset Question

The current important discussion is:

Since `trainings.csv` and `testing.csv` have the same structure, can we treat them as one larger labeled dataset?

The proposed approach:

```
trainings.csv
+
testing.csv
      ↓
Rename target columns consistently
      ↓
Combine datasets
      ↓
Remove exact duplicates
      ↓
Analyze disease frequency
      ↓
Split into train/validation/test
      ↓
Train model
      ↓
Evaluate
```

Approximate raw total:

`391 + 414 = 805 rows`

After removing 26 duplicates:

`approximately 779 unique rows`

This could be better than using only the 391-row training dataset.

However, we must first verify:

1. Are the 26 duplicates duplicates within `testing.csv` only?
2. Are any rows duplicated across training and testing?
3. How many examples does each disease have after combining?
4. Are the target labels exactly compatible?
5. Are all feature columns exactly identical and in the same order?

14. Important Data Leakage Warning

If we combine both datasets, we must remove duplicates **before splitting**.

Correct:

```
Training dataset
      +
Testing dataset
      ↓
Combine
      ↓
Remove duplicates
      ↓
Split
      ↓
Train / Validation / Test
```

Incorrect:

```
Split training.csv
      +
Use testing.csv for training
      ↓
Duplicate example appears in both train and test
      ↓
Artificially high accuracy
```

If the same exact symptom pattern appears in both the training and testing portions, the model may appear to perform extremely well simply because it has already seen the same example.

15. Current ML Strategy Recommendation

The next step should be a **dataset audit before changing the model**.

Do not immediately train another model.

First create a dataset audit that checks:

A. Exact shapes

```
train.shape  
test.shape
```

B. Column equality

```
list(train.columns) == list(test.columns)
```

Also:

```
set(train.columns) - set(test.columns)  
set(test.columns) - set(train.columns)
```

C. Duplicate rows

Within each dataset:

```
train.duplicated().sum()  
test.duplicated().sum()
```

Across combined data:

```
combined.duplicated().sum()
```

D. Disease distribution

After combining:

```
combined["Prognosis"].value_counts()
```

Then:

```
combined["Prognosis"].value_counts().describe()
```

And:

```
combined["Prognosis"].value_counts().value_counts().sort_index()
```

This tells us:

```
How many diseases have 1 example?  
How many have 2?  
How many have 3?  
...
```

This is extremely important.

16. Key ML Reality

The central issue is:

1326 possible symptoms
391 disease classes
approximately 779 unique combined examples

This is a:

High-dimensional
Low-sample
Multi-class classification problem

Therefore:

Simply increasing Random Forest parameters will not automatically solve the problem.

We need to determine whether the dataset itself contains enough examples for reliable classification.

The possible outcome may be:

- Combine and clean datasets
- Improve symptom normalization
- Train several baseline models
- Evaluate properly
- Select the best model
- Clearly document limitations

Potential baseline models, subject to project/PDF constraints:

Random Forest
Logistic Regression
Linear SVM
KNN

But only use technologies/models allowed by the project requirements.

17. Important Current Project Status

Completed

Backend

- FastAPI project setup
- Backend folder structure
- API application running
- Swagger documentation available

Database

- PostgreSQL installed and connected
- `medassist_ai` database created
- `users` table created
- `patient_profiles` table created
- `patient_symptoms` table created
- `disease_predictions` table created/used

Authentication

- Registration
- Login
- Password hashing
- JWT authentication
- Protected endpoints

Patient module

- Create patient profile
- Add symptoms
- View symptoms
- Delete symptoms
- Disease prediction endpoint
- Available symptom endpoint

ML

- Dataset loading
- Dataset inspection
- Initial Random Forest model
- Model saved using Joblib
- Feature names saved
- Prediction service integrated with FastAPI

18. Current Problems

Problem 1 — ML predictions are unreliable

Examples:


```
fever → gout  
low blood pressure → epilepsy
```

The API works, but the model output is not yet reliable.

Problem 2 — Training dataset has very few examples per disease

Current approximate situation:

```
391 training rows  
391 unique diseases
```

This must be investigated carefully.

Problem 3 — Symptom names are inconsistent

Examples:

```
fever  
Fever  
Fever-- usually low-grade
```

and:

```
Fatigue  
Fatigue(Feeling Tired)  
Fatigue, reduced exercise ability
```

This makes exact user input matching unreliable.

Problem 4 — Training/testing datasets need proper comparison

Although they appear to have the same structure, we must verify:

- Exact columns
- Exact order
- Exact data types
- Target label compatibility
- Duplicate rows
- Cross-dataset duplicates

Problem 5 — The third patient-profile dataset is different

The 349×10 dataset should not be merged blindly with the 1326-feature datasets.

We need to understand its role according to the project PDF.

19. Current Milestone 1 Remaining Work

The remaining work is broadly:

1. Finalize dataset strategy

Determine:

Which dataset is used for what?

Potentially:

`trainings.csv + testing.csv`

↓

Disease prediction dataset

Patient profile dataset

↓

Patient/health outcome or risk-related functionality

But this must be confirmed against the project PDF.

2. Complete dataset preprocessing

Potential tasks:

- Remove exact duplicates
 - Verify missing values
 - Standardize columns
 - Normalize symptom names
 - Validate disease labels
 - Analyze class distribution
-

3. Improve disease prediction pipeline

Pipeline:

```
Patient symptoms
↓
Symptom normalization
↓
Feature mapping
↓
Feature vector creation
↓
ML model
↓
Disease prediction
```

4. Complete prediction history API

Expected endpoint:

```
GET /patient/prediction-history
```

This should return previous predictions for the authenticated patient.

5. Implement risk assessment

The system should provide an appropriate risk level based on the project requirements.

Possible structure:

```
Prediction
↓
Risk assessment
↓
Risk level
```

Do not invent a medical risk model outside the project requirements.

6. Implement recommendations

Recommendations should be based on the project's specified logic.

The system should not present the ML prediction as a confirmed medical diagnosis.

7. Frontend-backend integration

Expected workflow:

```
User
↓
Login
↓
Patient dashboard
↓
Patient profile
↓
Enter/select symptoms
↓
Submit symptoms
↓
Backend prediction
↓
Risk assessment
↓
Recommendations
↓
Prediction history
```

8. End-to-end testing

Test:

```
Registration
↓
Login
↓
JWT authorization
↓
Create profile
↓
Add symptoms
↓
Predict disease
↓
Save prediction
↓
Retrieve history
```

20. Suggested Immediate Next Step

The very next thing to do in the new chat should be:

Perform a complete audit of `trainings.csv` and `testing.csv` together before changing the ML model.

The audit should:

1. Load both CSV files correctly.
2. Confirm exact column equality.

3. Rename the testing target column to a common name.
4. Check duplicate rows in each dataset.
5. Check duplicates across both datasets.
6. Combine them.
7. Remove exact duplicates.
8. Print the disease frequency distribution.
9. Check the number of diseases with 1, 2, 3, etc. examples.
10. Decide whether the combined dataset is suitable for a train/validation/test split.

Only after this should we decide the final training strategy.

Copy this as the opening message in the new chat

You can paste the following:

I am continuing my MedAssist AI project from another chat. Please use this context as the project orientation.

The project is a healthcare assistance system with a FastAPI backend, PostgreSQL database, patient module, authentication, ML disease prediction, risk assessment, recommendations, and frontend integration. We must use only technologies, datasets, and modules specified in the project PDF.

Current backend:

backend/app/

Important files:

- main.py
- database.py
- auth.py
- patient.py
- prediction_service.py
- train_disease_model.py

PostgreSQL database:

medassist_ai

Existing tables:

- users
- patient_profiles
- patient_symptoms
- disease_predictions

Authentication and patient profile/symptom APIs are working. Disease prediction is integrated with FastAPI, but the model produces unreliable predictions such as fever → gout and low blood pressure → epilepsy.

Datasets:

1. `trainings.csv`
 - 391 rows
 - 1327 columns
 - 1326 symptom features
 - target `Prognosis`
 - 391 unique diseases
2. `testing.csv`
 - approximately 414 data rows
 - same feature structure as training dataset
 - target appears as `Prognosis` (prediction)
 - contains 26 duplicate entries
3. `Disease_symptom_and_patient_profile_dataset.csv`
 - 349 rows
 - 10 columns
 - columns:
Disease, Fever, Cough, Fatigue, Difficulty Breathing, Age, Gender, Blood Pressure, Cholesterol Level, Outcome Variable

We are currently considering treating `trainings.csv` and `testing.csv` as two labeled datasets, removing exact duplicates, combining them, and then creating a proper train/validation/test split. Before doing that, I want to audit:

- exact column equality
- target label compatibility
- duplicate rows within each dataset
- duplicate rows across datasets
- disease frequency after combining
- number of diseases with only 1, 2, 3, etc. examples

Do not jump directly to changing the model. First help me perform a proper dataset audit step by step, clearly and slowly. After the audit, we will decide whether combining the datasets and retraining is actually valid.

Also, please remember that the symptom names are inconsistent, with examples such as:

- fever
- Fever
- Fever-- usually low-grade
- Fatigue(Feeling Tired)

- Fatigue, reduced exercise ability

So symptom normalization/mapping is another important part of the ML pipeline.

My immediate goal is to complete Milestone 1 and prepare the final working patient workflow and presentation.